



Chapter 4

CPU SCHEDULING

Đinh Công Đoàn

1



Content

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Multiple-Processor Scheduling
- Real-Time Scheduling
- Algorithm Evaluation

Đinh Công Đoàn

2



Basic concepts

- Maximum CPU utilization obtained with multiprogramming
- CPU–I/O Burst Cycle – Process execution consists of a cycle of CPU execution and I/O wait.
- CPU burst distribution

Đinh Công Đoàn

3



CPU scheduler

- Selects from among the processes in memory that are ready to execute, and allocates the CPU to one of them.
- CPU scheduling decisions may take place when a process:
 - ✓ 1. Switches from running to waiting state.
 - ✓ 2. Switches from running to ready state.
 - ✓ 3. Switches from waiting to ready.
 - ✓ 4. Terminates.
 - ✓ Scheduling under 1 and 4 is nonpreemptive.
 - ✓ All other scheduling is preemptive

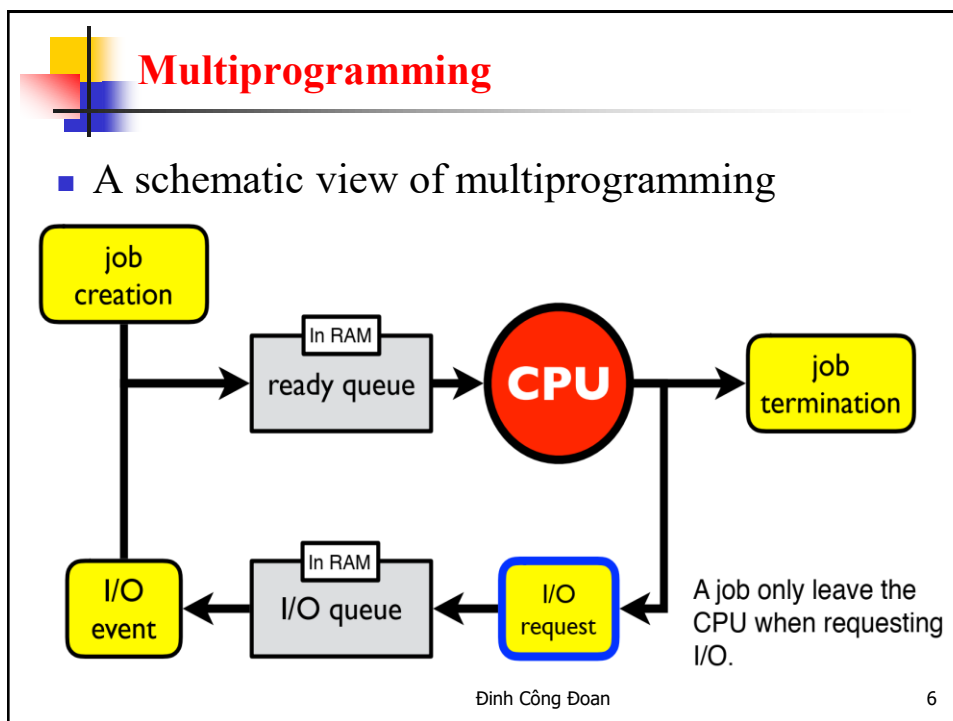
Đinh Công Đoàn

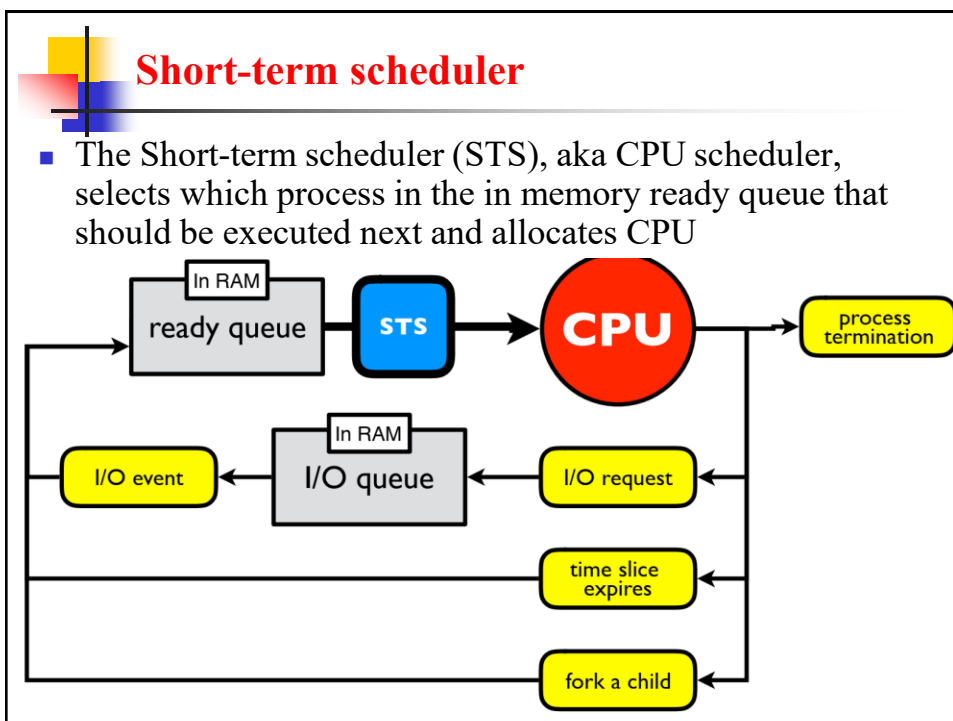
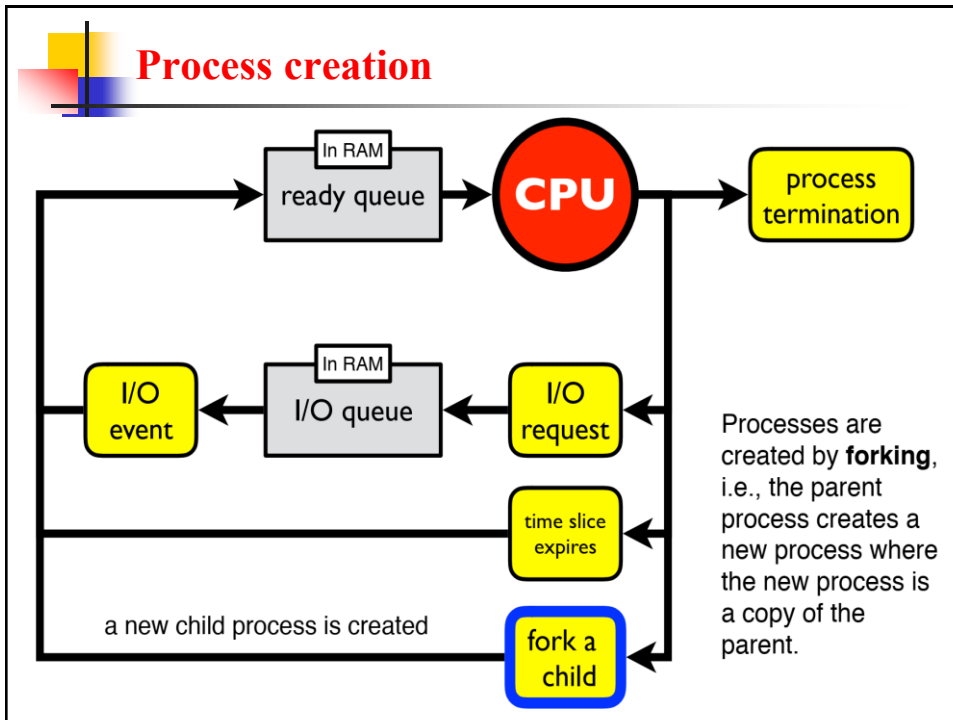
4

Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the shortterm scheduler; this involves:
 - ✓ switching context
 - ✓ switching to user mode
 - ✓ jumping to the proper location in the user program to restart that program
- Dispatch latency – time it takes for the dispatcher to stop one process and start another running.

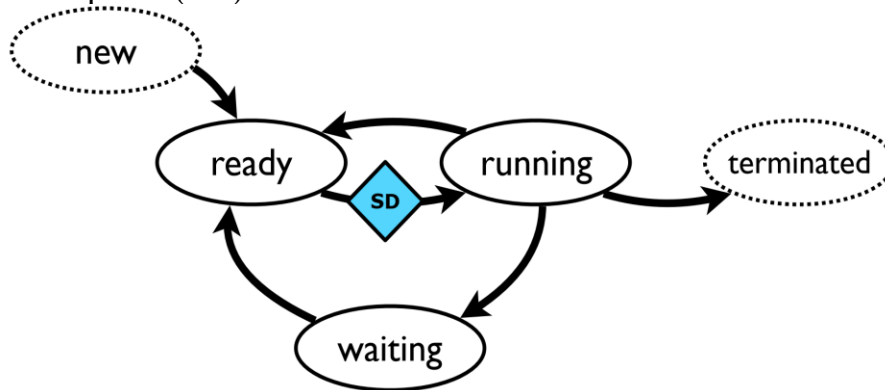
Đinh Công Đoàn 5





Scheduler dispatch

- The CPU scheduler selects one process from among the processes in memory that are READY to execute. The scheduler dispatcher then gives the selected process control of the CPU. This action is called scheduler dispatch (SD).



Basic concepts

- Maximum CPU utilization obtained with multiprogramming
- CPU-I/O Burst Cycle – Process execution consists of a cycle of CPU execution and I/O wait.
- CPU burst distribution

load store
add store
read from file

CPU burst

wait for I/O

I/O burst

store increment
index
write to file

CPU burst

wait for I/O

I/O burst

load store
add store
read from file

CPU burst

wait for I/O

I/O burst

⋮

CPU bursts and I/O bursts

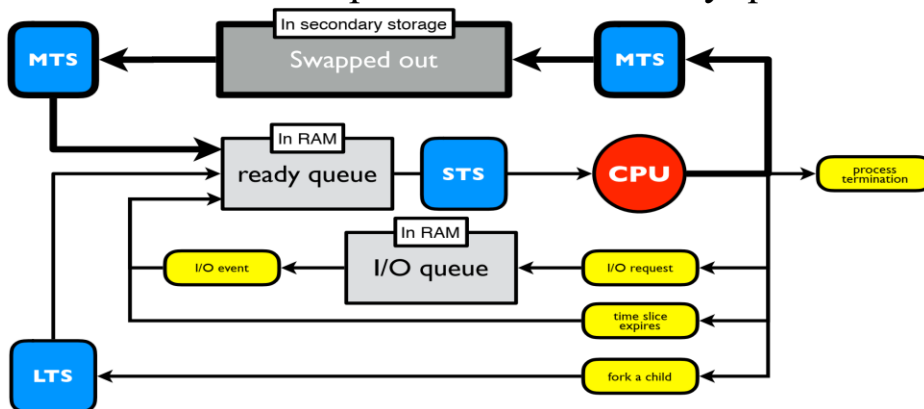
- An **I/O-bound** process spends more time doing I/O than computations and is characterized by **many short CPU bursts**
- An **CPU-bound** process spends more time doing computations and is characterised by **few very long CPU bursts**

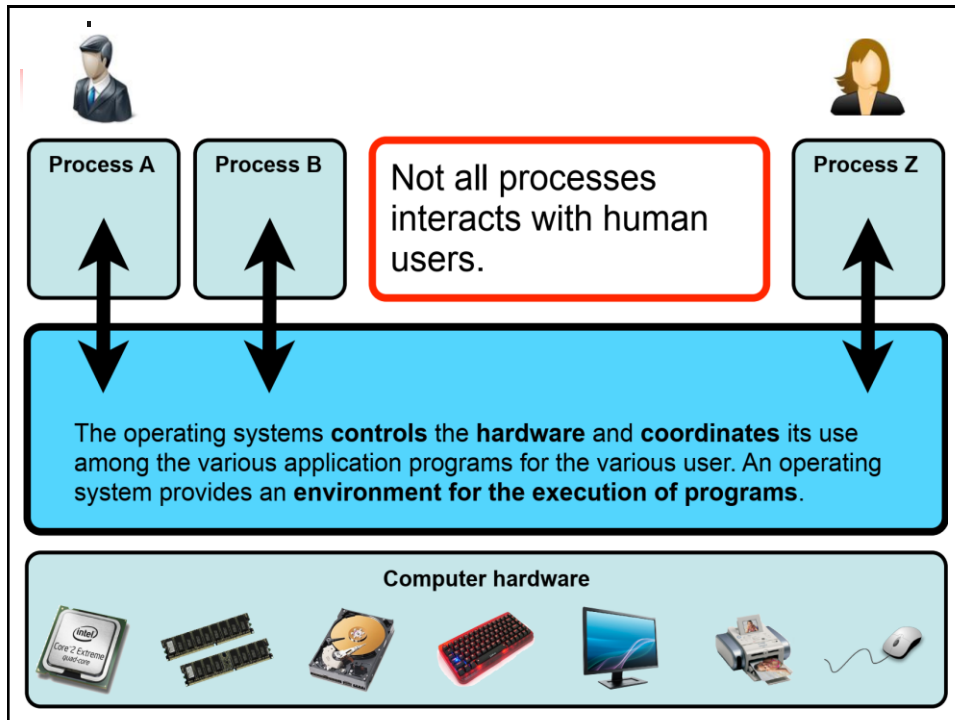


Đinh Công Đoàn

I/O bound and CPU bound processes

- The medium-term scheduler (MTS) can be used to maintain a good balance between I/O bound and CPU bound processes in the ready queue





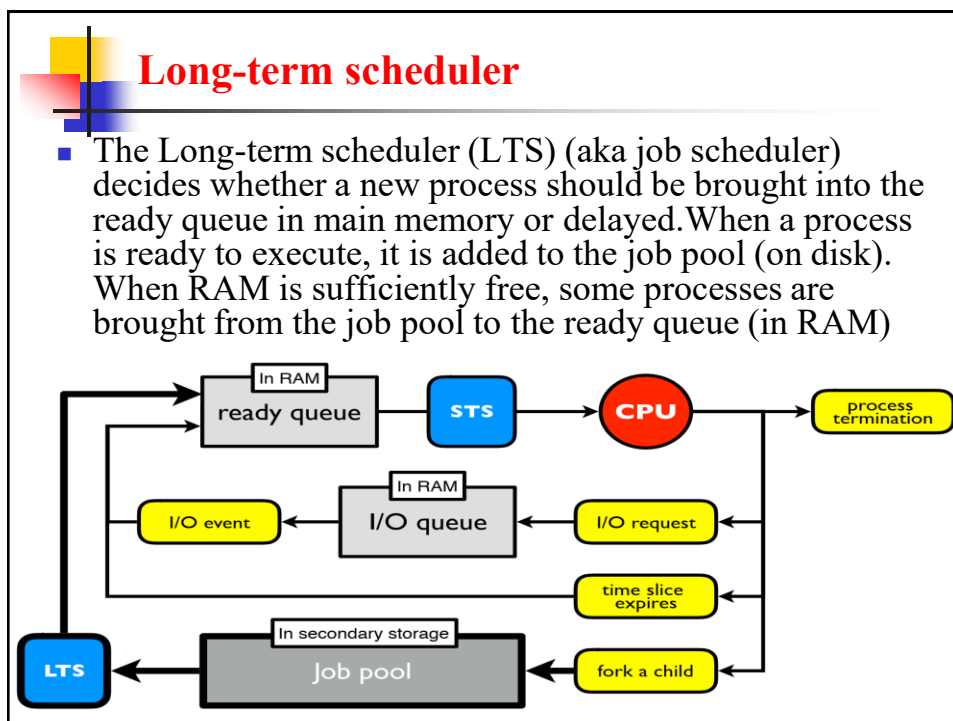
Scheduling Objectives

- Enforcement of fairness in allocating resources to processes
- Enforcement of priorities
- Make best use of available system resources
- Give preference to processes holding key resources.
- Give preference to processes exhibiting good behavior.
- Degrade gracefully under heavy loads

Program Behavior Issues

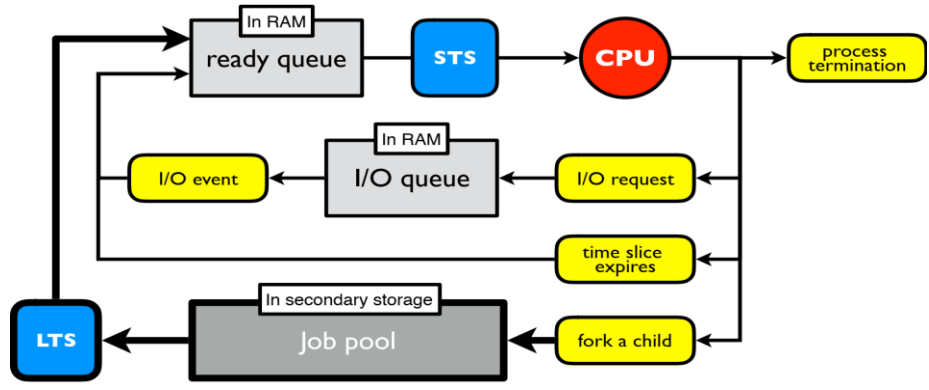
- I/O boundedness
 - ✓ short burst of CPU before blocking for I/O
- CPU boundedness
 - ✓ extensive use of CPU before blocking for I/O
- Urgency and Priorities
- Frequency of preemption
- Process execution time
- Time sharing
 - ✓ amount of execution time process has already received

Đinh Công Đoàn 15



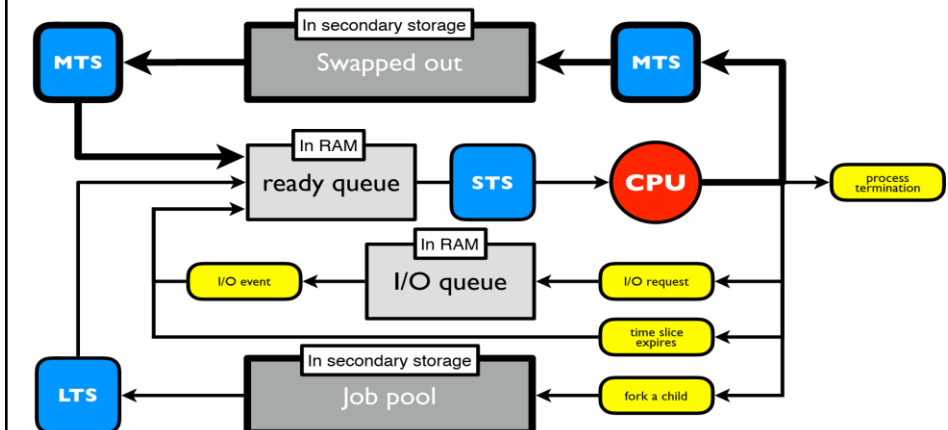
Long-term scheduler

- On some systems, the long-term scheduler may be absent or minimal. For example, time-sharing systems such as UNIX and Microsoft Windows systems often have no long-term scheduler but simply put every new process in memory for the short-term scheduler.



Medium-term scheduler

- The medium-term scheduler (MTS) temporarily removes processes from main memory and places them in secondary storage and vice versa, which is commonly referred to as "swapping in" and "swapping out".





Levels of Scheduling

- High Level / Job Scheduling
 - ✓ Selects jobs allowed to compete for CPU and other system resources.
- Intermediate Level /Medium Term Scheduling
 - ✓ Selects which jobs to temporarily suspend/resume to smooth fluctuations in system load.
- Low Level (CPU) Scheduling or Dispatching
 - ✓ Selects the ready process that will be assigned the CPU.
 - ✓ Ready Queue contains PCBs of processes.

Đinh Công Đoàn 19



CPU Scheduler

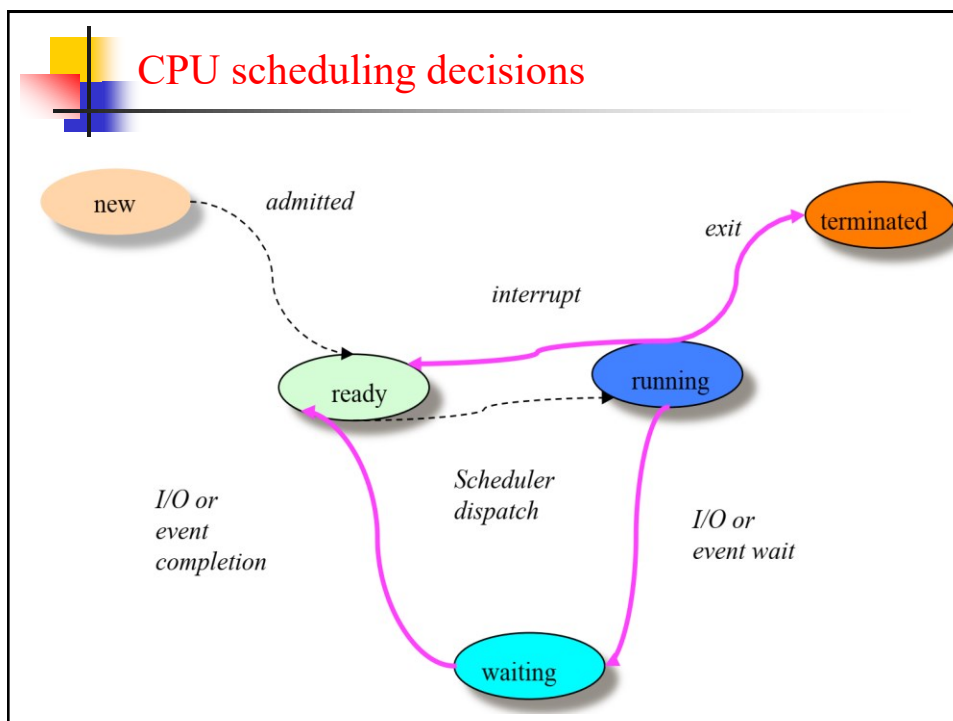
- Selects from among the processes in memory that are ready to execute, and allocates the CPU to one of them.
 - ✓ Non-preemptive Scheduling
 - Once CPU has been allocated to a process, the process keeps the CPU until
 - Process exits OR
 - Process switches to waiting state
 - ✓ Preemptive Scheduling
 - Process can be interrupted and must release the CPU.
 - Need to coordinate access to shared data

Đinh Công Đoàn 20

CPU Scheduling Decisions

- CPU scheduling decisions may take place when a process:
 - ✓ 1. switches from running state to waiting state
 - ✓ 2. switches from running state to ready state
 - ✓ 3. switches from waiting to ready
 - ✓ 4. terminates
- Scheduling under 1 and 4 is nonpreemptive.
- All other scheduling is preemptive.

Đinh Công Đoàn 21





Scheduling Criteria

- CPU Utilization
 - ✓ Keep the CPU and other resources as busy as possible
- Throughput
 - ✓ # of processes that complete their execution per time unit.
- Turnaround time
 - ✓ amount of time to execute a particular process from its entry time
- Waiting time
 - ✓ amount of time a process has been waiting in the ready queue.
- Response Time (in a time-sharing environment)
 - ✓ amount of time it takes from when a request was submitted until the first response is produced, NOT output

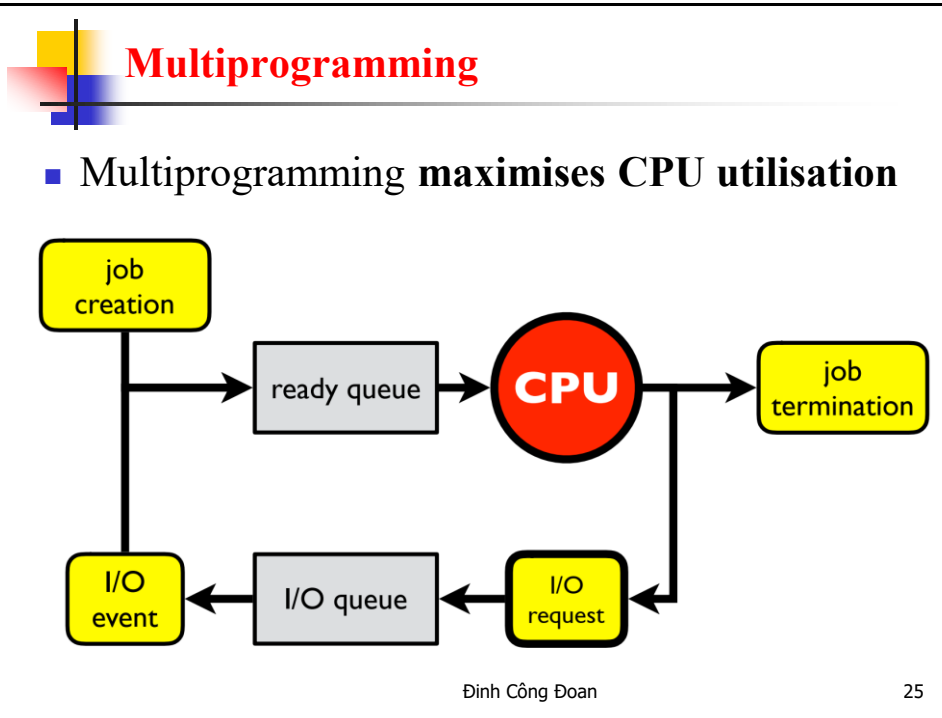
Đinh Công Đoàn 23



Optimization Criteria

- Optimize overall system: keep the CPU as busy as possible
 - ✓ Max CPU Utilization
 - ✓ Max Throughput
- Optimize individual processes' performance
 - ✓ Min Turnaround time
 - ✓ Min Waiting time
 - ✓ Min Response time

Đinh Công Đoàn 24



Scheduling criteria

Criteria	Definition	Goal
CPU utilization	The % of time the CPU is executing user level process code.	Maximize
Throughput	Number of processes that complete their execution per time unit.	Maximize
Turnaround time	Amount of time to execute a particular process.	Minimize
Waiting time	Amount of time a process has been waiting in the ready queue.	Minimize
Response time	Amount of time it takes from when a request was submitted until the first response is produced.	Minimize



Classification of processes

- In general, processes can be classified by the following characteristics.
 - ✓ Interactive
 - ✓ Batch
 - ✓ Real-Time
 - ✓ I/O Bound
 - ✓ CPU Bound

Đinh Công Đoàn 27



Interactive

- Interactive processes interact constantly with their human users.
 - ✓ Spend a lot of time waiting for keypresses and mouse operations.
 - ✓ When input is received, the process must be woken up quickly, or the user will find the system to be unresponsive.
 - ✓ Typically, the average delay must fall between 50 and 150 ms. The variance of such delay must also be bounded, or the user will find the system to be erratic.
- Typical examples:
 - ✓ Command shells and interpreters, text editors, graphical applications and games

Đinh Công Đoàn 28



Batch

- Batch processes do not interact with human users.
 - ✓ Do not need to be responsive.
 - ✓ Often run in the background.
 - ✓ Often penalised by the scheduler.
- Typical examples:
 - ✓ Compilers, database search engines, and scientific computations.

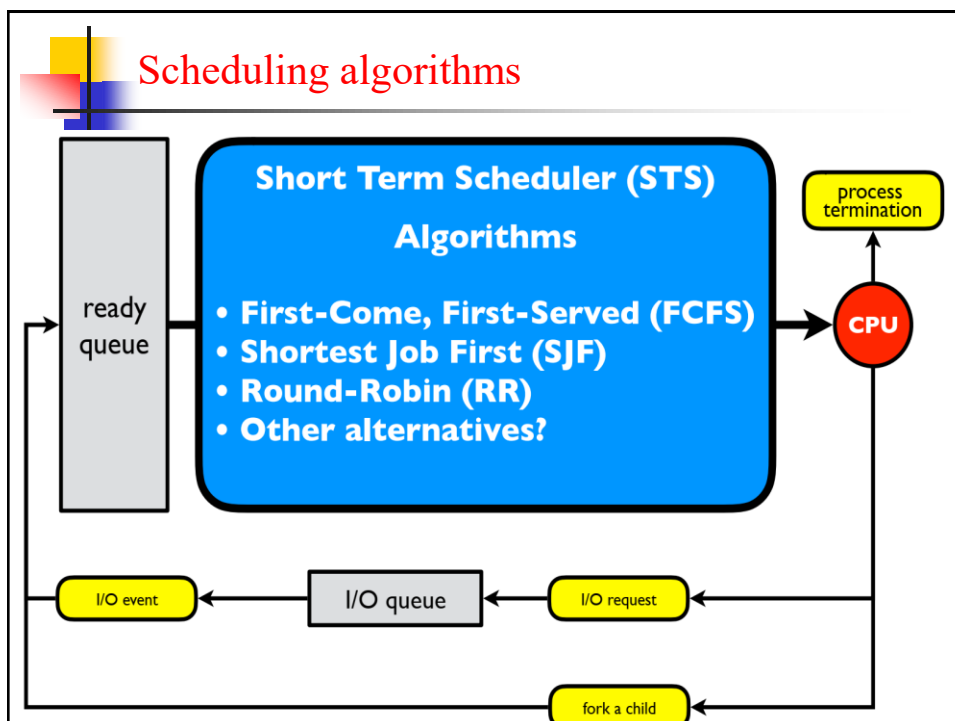
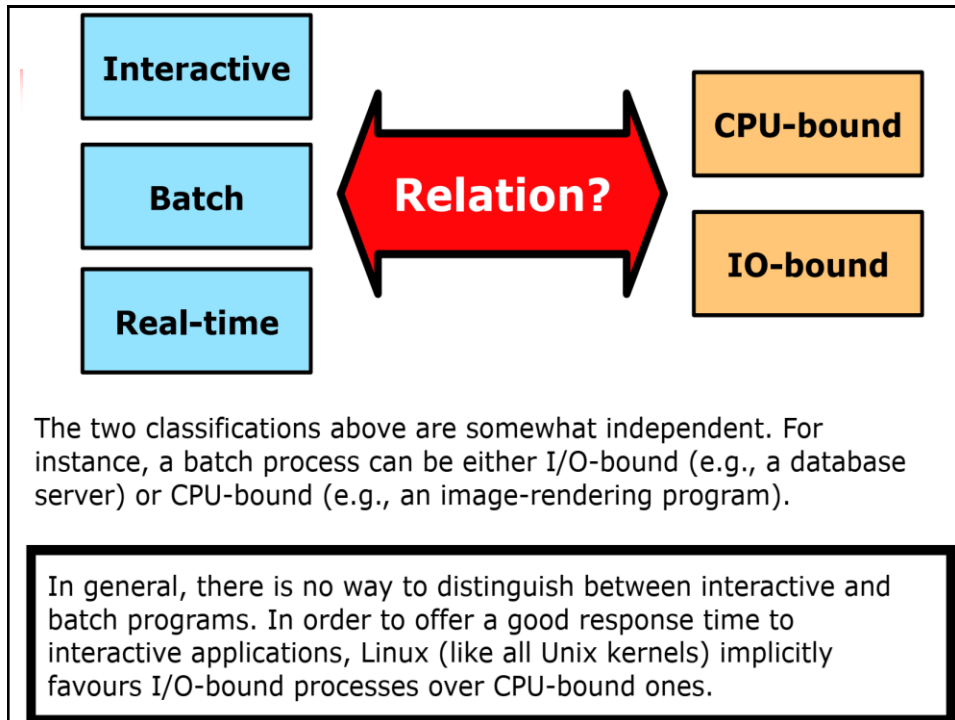
Đinh Công Đoàn 29

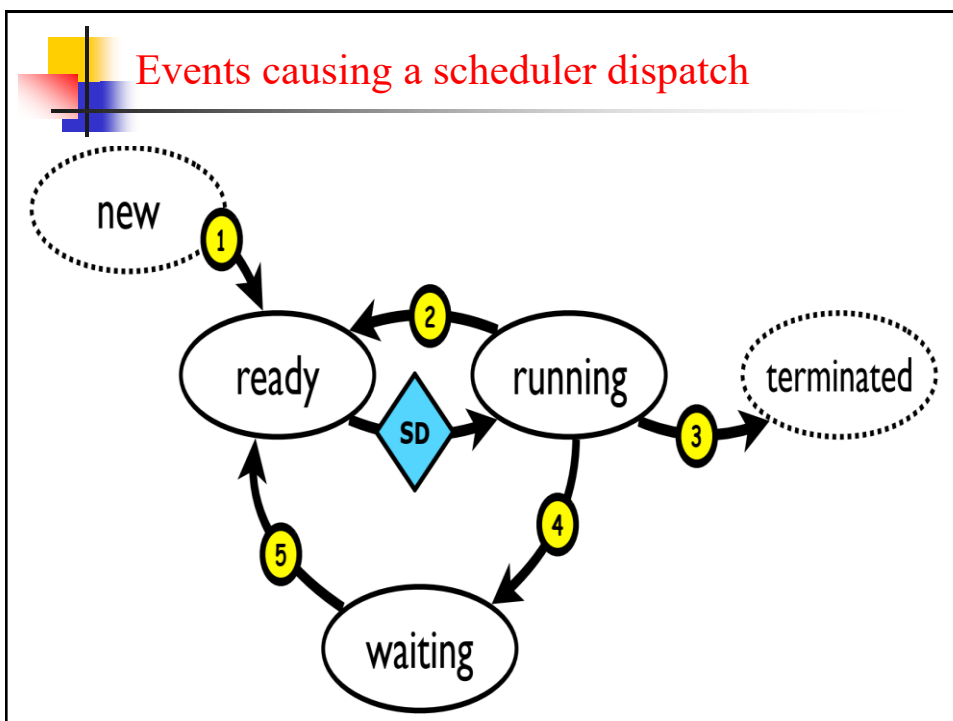
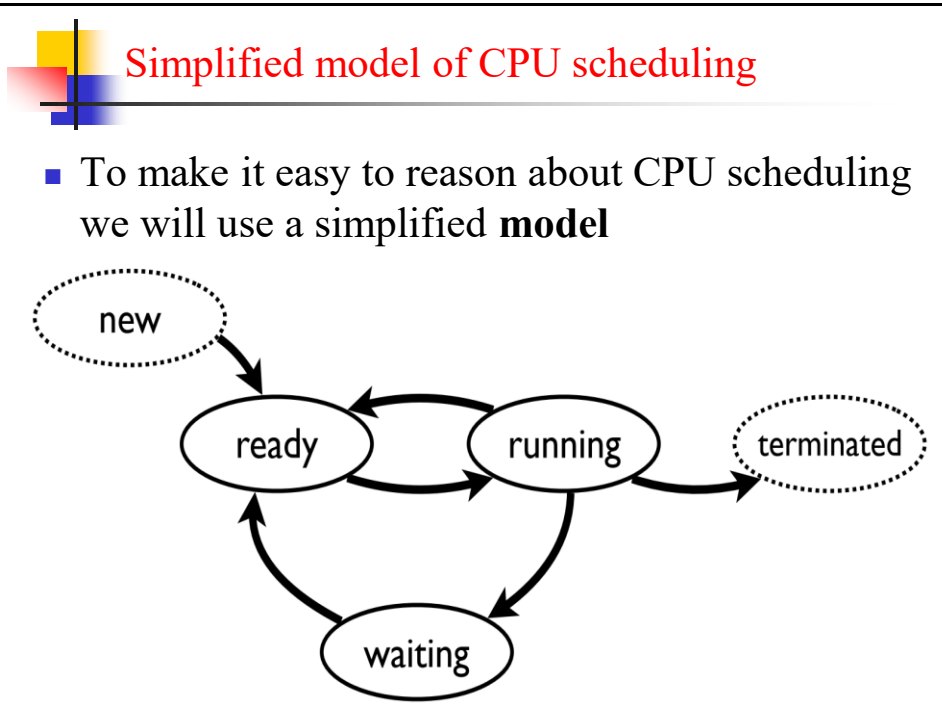


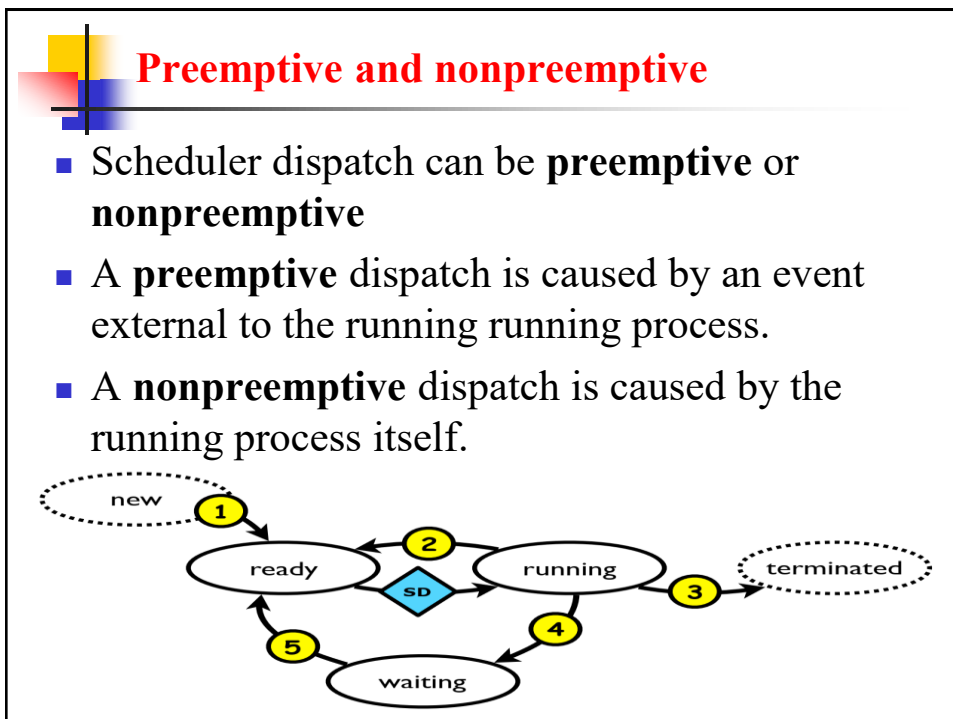
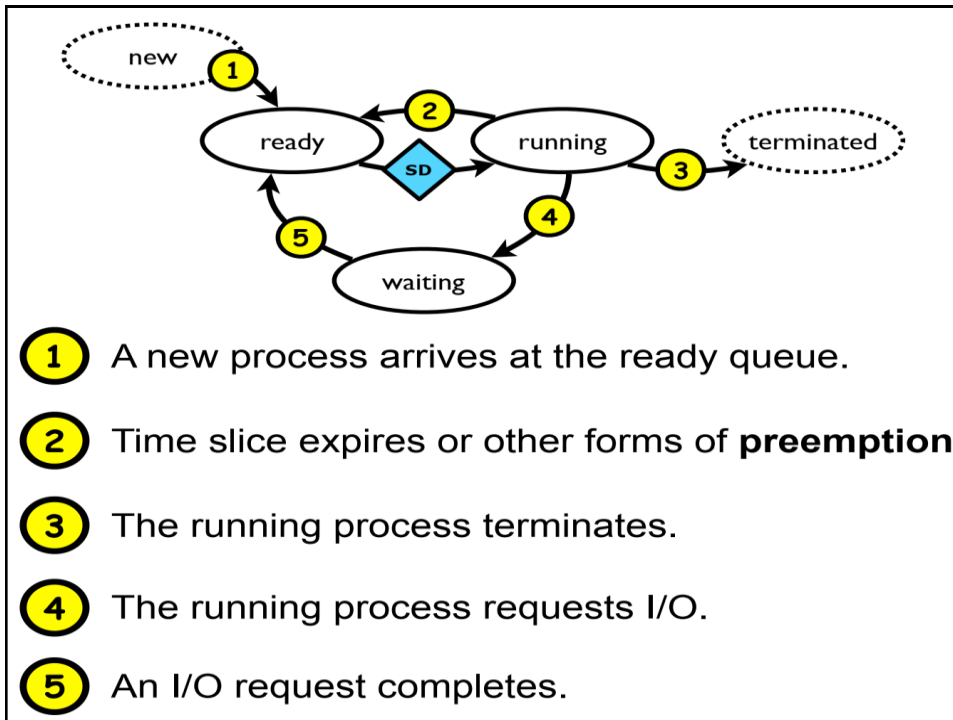
Real-time

- Real-time processes have very strong scheduling requirements.
 - ✓ Such processes should never be blocked by lowerpriority processes.
 - ✓ Should have a short response time.
 - ✓ Most important, response time should have a minimum variance.
- Typical examples:
 - ✓ Video and sound applications, robot controllers, and programs that collect data from physical sensors.

Đinh Công Đoàn 30

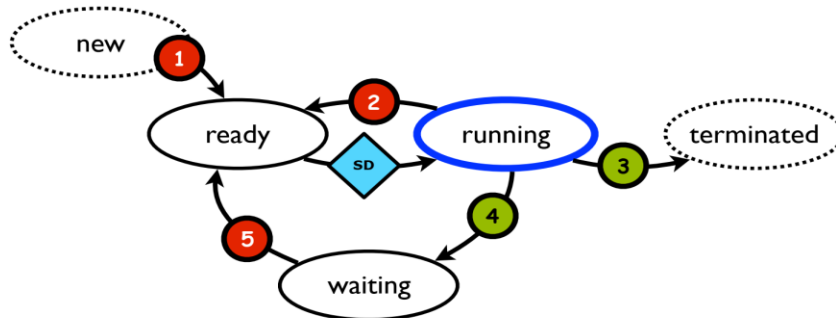






Preemptive and nonpreemptive

- Scheduler dispatch can be **preemptive** or **nonpreemptive**.



Events causing a **preemptive** scheduler dispatch:



Events causing a **nonpreemptive** scheduler dispatch:

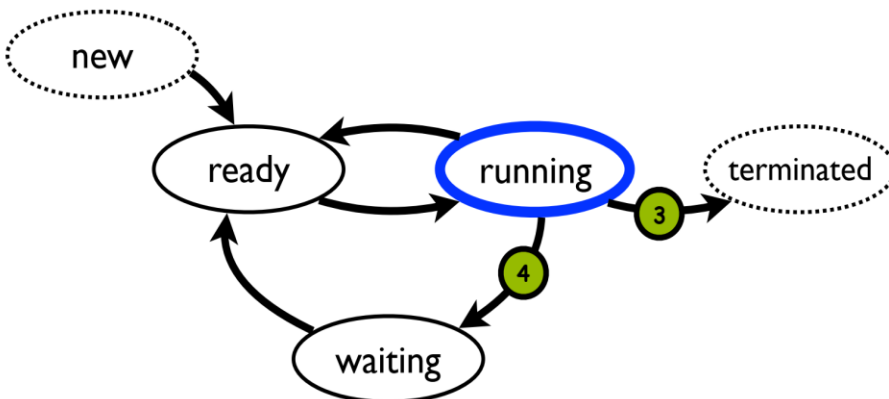


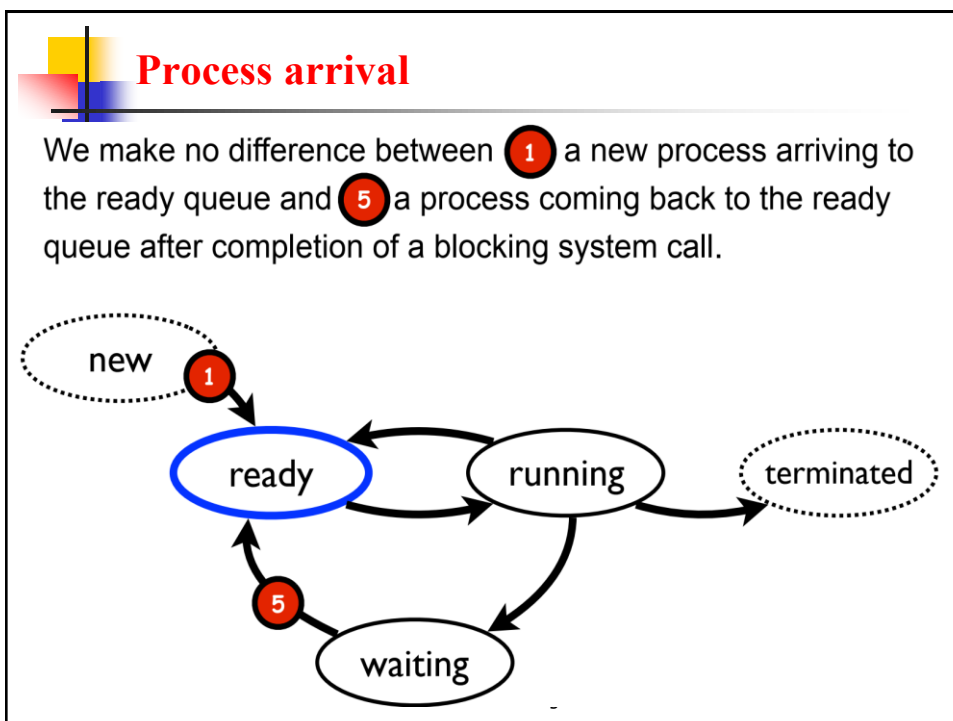
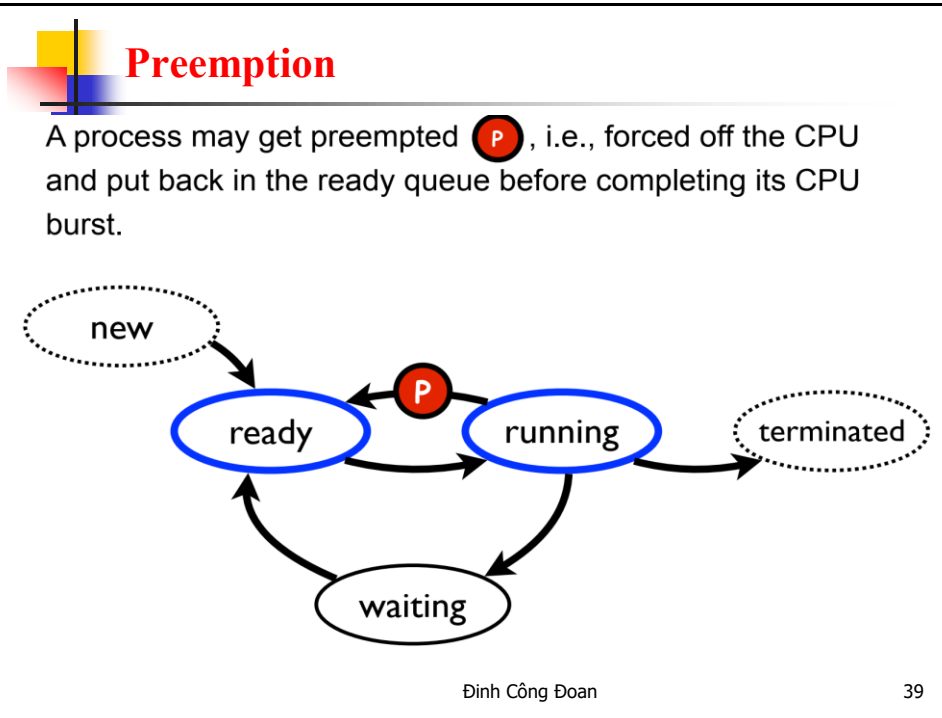
Đinh Công Đoàn

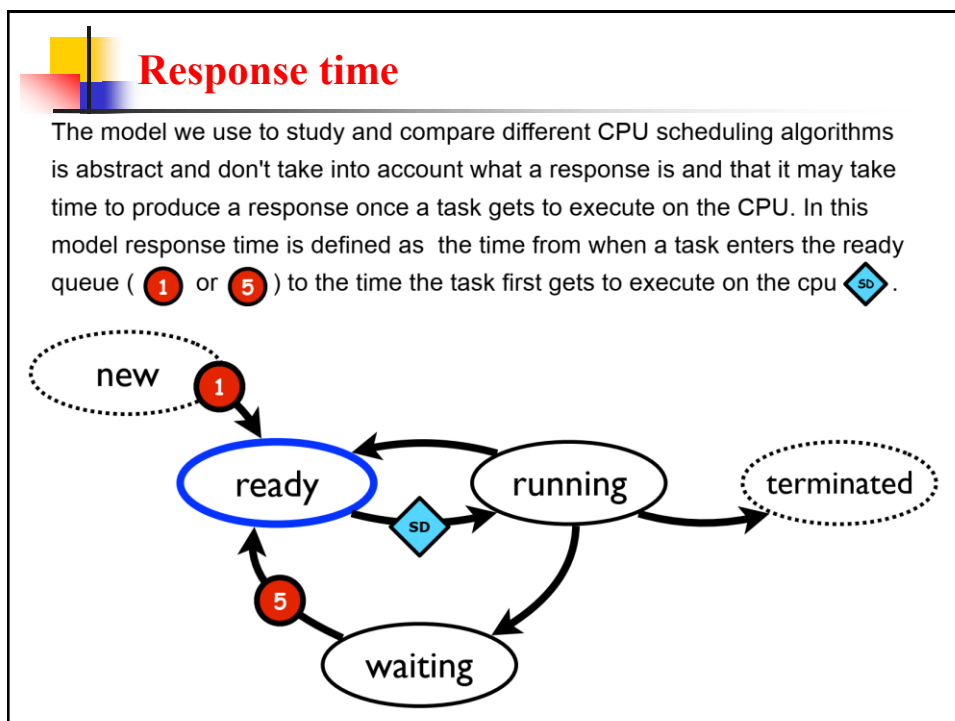
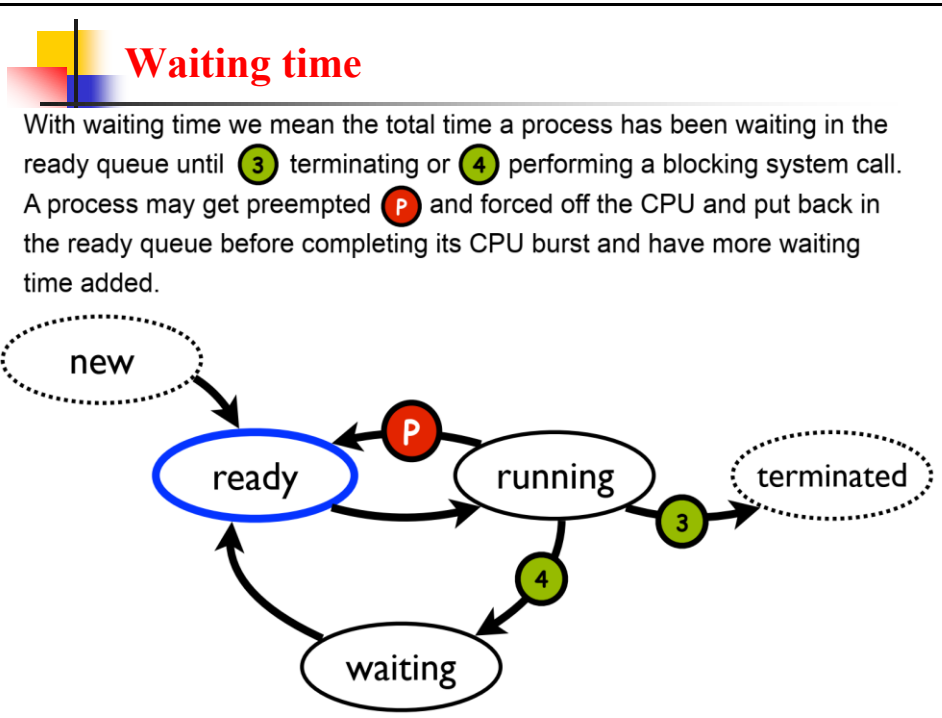
37

CPU burst

With CPU burst we mean the time spent by a running process using the CPU before ③ terminating or ④ performing a blocking system call.

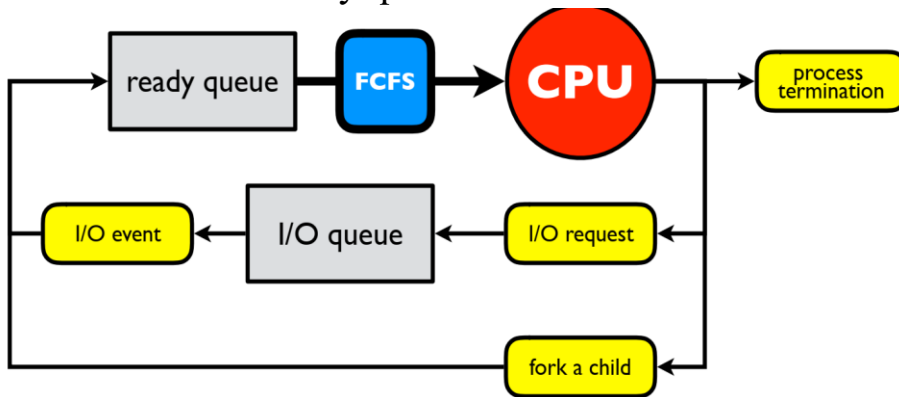




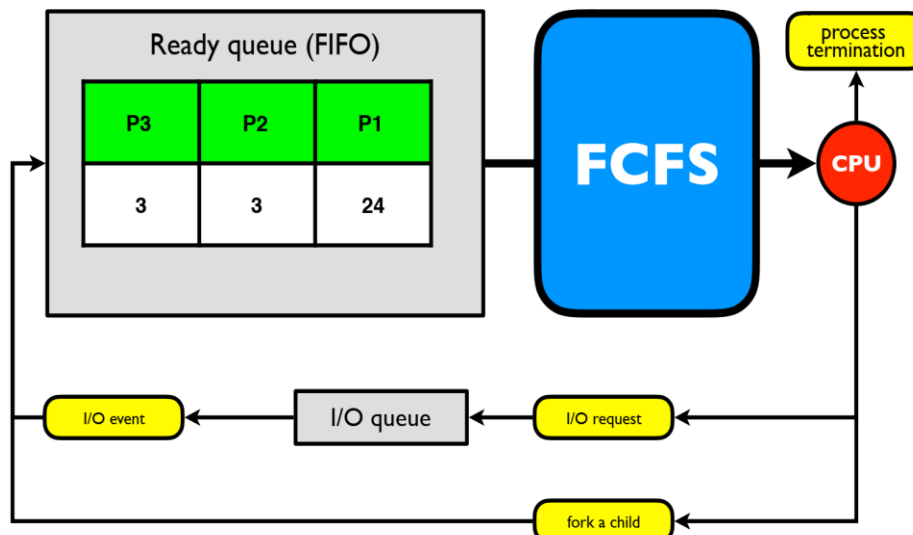


First-Come, First-Served (FCFS)

- The first come, first served (commonly called FIFO – first in, first out) process scheduling algorithm is the simplest process scheduling algorithm. Processes are executed on the CPU in the same order they arrive to the ready queue

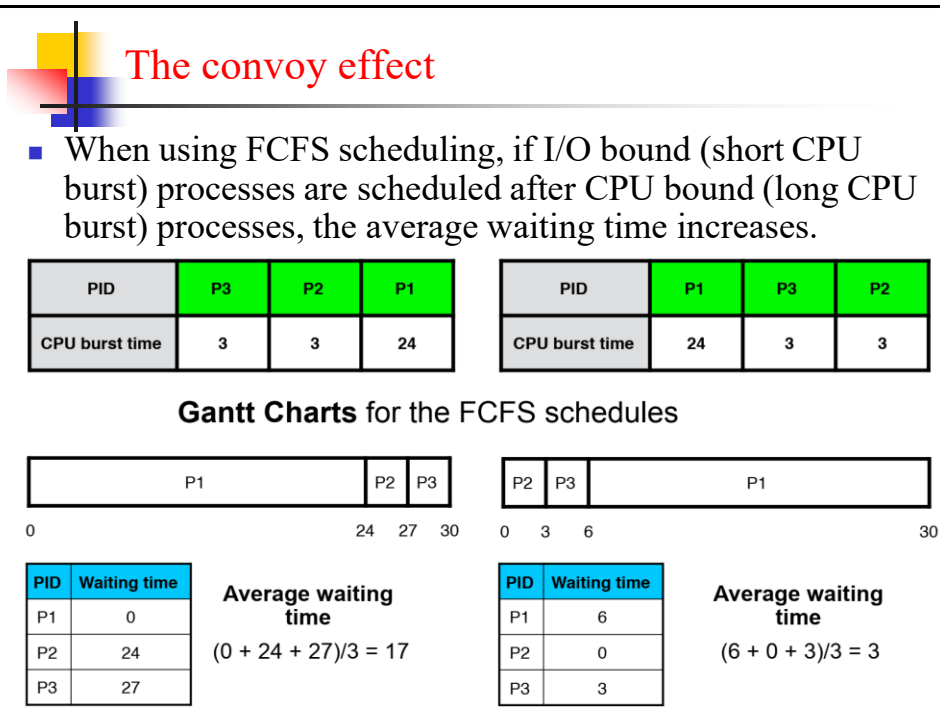


Scheduling algorithms



Đinh Công Đoàn

44



First Come First Serve (FCFS) Scheduling

- Policy: Process that requests the CPU FIRST is allocated the CPU FIRST.
 - ✓ FCFS is a non-preemptive scheduling algorithm.
- Implementation - using FIFO queues
 - ✓ incoming process is added to the tail of the queue.
 - ✓ Process selected for execution is taken from head of queue.
- Performance metric - Average waiting time in queue.
- Gantt Charts are used to visualize schedules

Đinh Công Đoàn 48

Example

Process	Burst Time
P1	24
P2	3
P3	3

- Suppose that the processes arrive in the order: P1 , P2 , P3. The Gantt Chart for the schedule is:

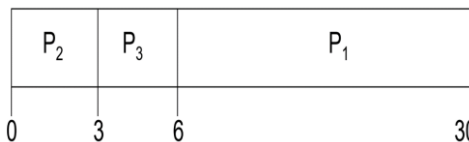


Đinh Công Đoàn

49

Example (Cont)

- Waiting time
 - ✓ $P1 = 0$;
 - ✓ $P2 = 24$;
 - ✓ $P3 = 27$;
- Average waiting time
 $(0+24+27)/3 = 17$
- Suppose that the processes arrive in the order P2 , P3 , P1 .
- The Gantt chart for the schedule is:
- Waiting time for P1 = 6; P2 = 0; P3 = 3
- Average waiting time:
 - $(6 + 0 + 3)/3 = 3$
- Much better than previous case



Đinh Công Đoàn

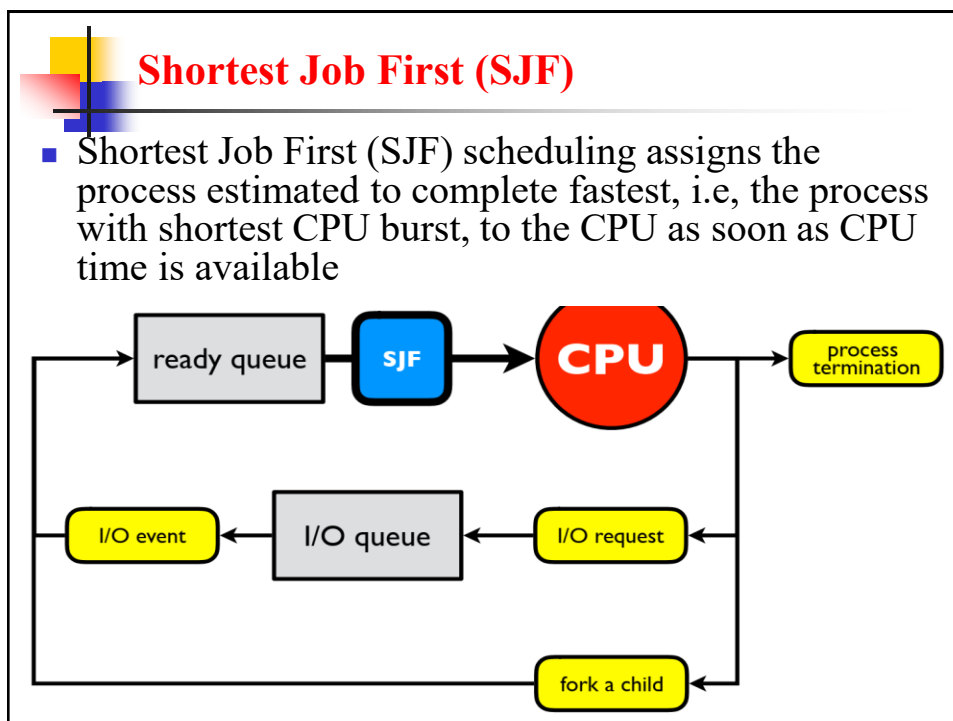
50



First-Come, First-Served (FCFS)

- Advantages
 - ✓ Simple
 - ✓ Easy
 - ✓ First come, first served
- Disadvantages
 - ✓ This scheduling method is **nonpreemptive**, that is, a process will execute its CPU burst until it finishes.
 - ✓ Because of this nonpreemptive scheduling, short processes which are at the back of the queue have to wait for the long process at the front to finish making the average waiting time increase - the **convoy effect**

Đinh Công Đoàn 51





Shortest Job First (SJF)

- Shortest Job First (SJF) scheduling assigns the process estimated to complete fastest to the CPU as soon as CPU time is available.
 - ✓ Associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest burst time.
 - ✓ Also known as Shortest Process Next (SPN) scheduling.
 - ✓ Also known as Shortest job next (SJN) scheduling.
 - ✓ SJF is **optimal** – gives **minimum average waiting** time for a given set of processes

Đinh Công Đoàn

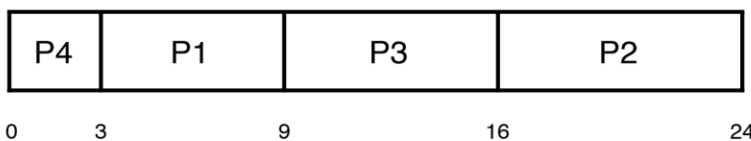
53

PID	P4	P3	P2	P1
CPU burst time	3	7	8	6

Estimated values of CPU bursts

Processes in the ready queue.

Gantt Chart for the SJF schedule

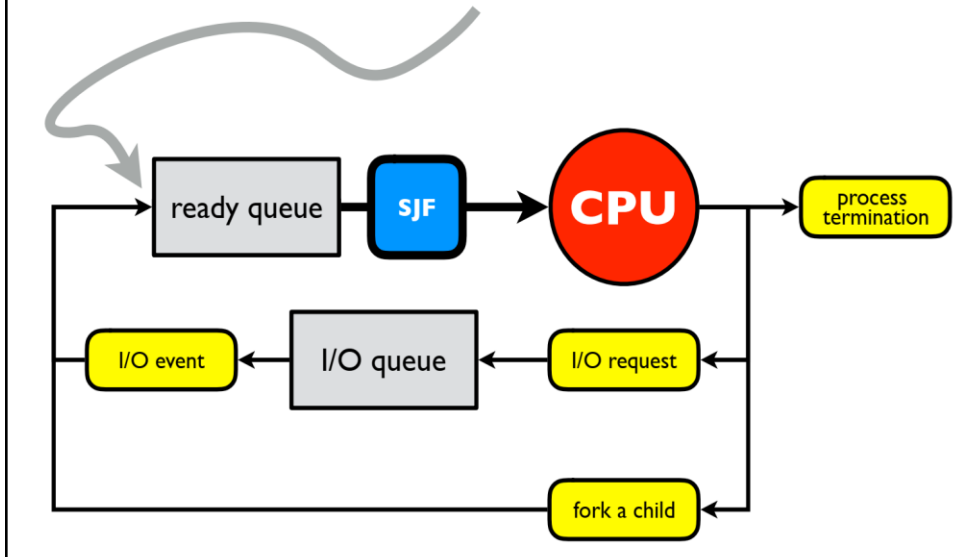


Process	Waiting time
P1	3
P2	16
P3	9
P4	0

Average waiting time

$$(3 + 16 + 9 + 0)/4 = 28/4 = 7$$

But does all processes arrive at the same time to the ready queue?

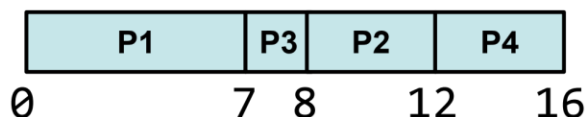


Example of SJF

Ready queue		
Process	Arrival time	Burst Time
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Time	Action
0	Only P1 ready to execute
7	When P1 finish, the process in the ready queue with the shortest burst time is selected for dispatch, in this example P3.
8	When P3 finish, both P2 and P4 have burst time 4. Use FIFO to break ties. In this example P2 arrives before P4, hence P2 is selected for execution
12	When P2 finish, only P4 remains in the ready queue.
16	The ready queue is now empty, all processes done.

Gantt Chart for the SJF schedule



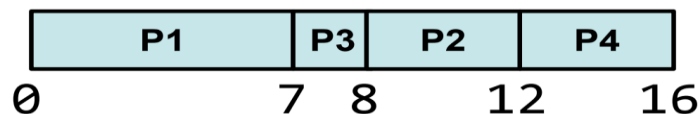
Example of SJF - Average waiting time

Ready queue			SJF				
Process	Arrival time	Burst Time	T _{dispatch}	-	T _{arrival}	=	Waiting time
P1	0	7	0	-	0	=	0
P2	2	4	8	-	2	=	6
P3	4	1	7	-	4	=	3
P4	5	4	12	-	5	=	7

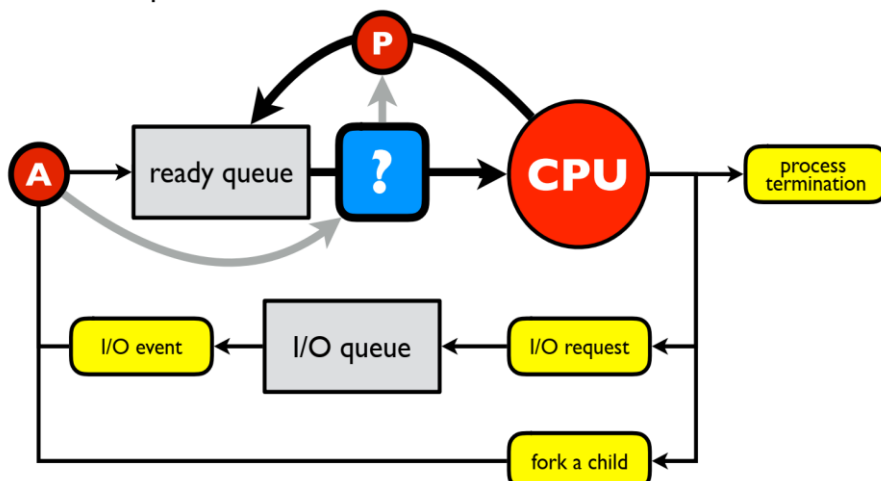
Average waiting time

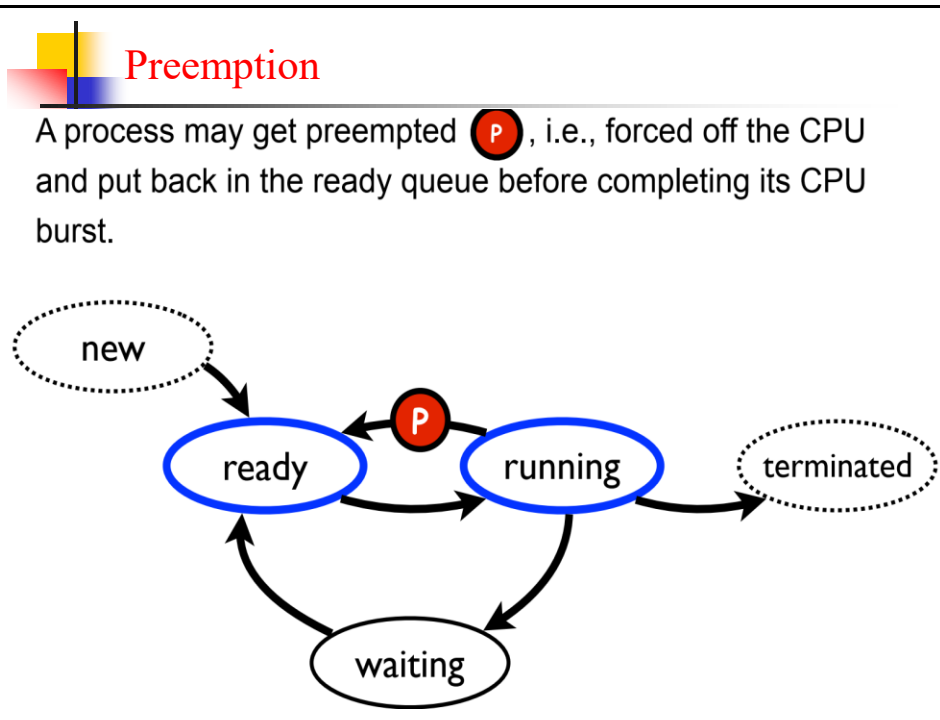
$$(0 + 6 + 3 + 7)/4 = 16/4 = 4$$

Gantt Chart for the SJF schedule



- A** When a process with a shorter burst time compared to the currently scheduled process arrives to the ready queue ...
- P** ... wouldn't it be more optimal (minimising the average waiting time) to preempt the running process and switch to newly arrived process?

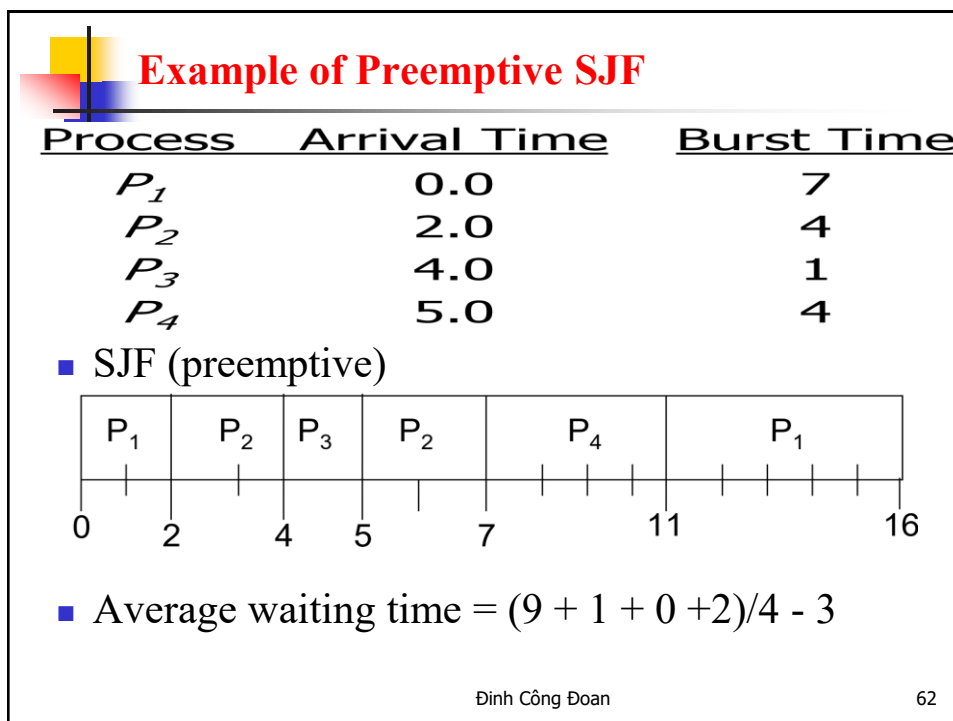
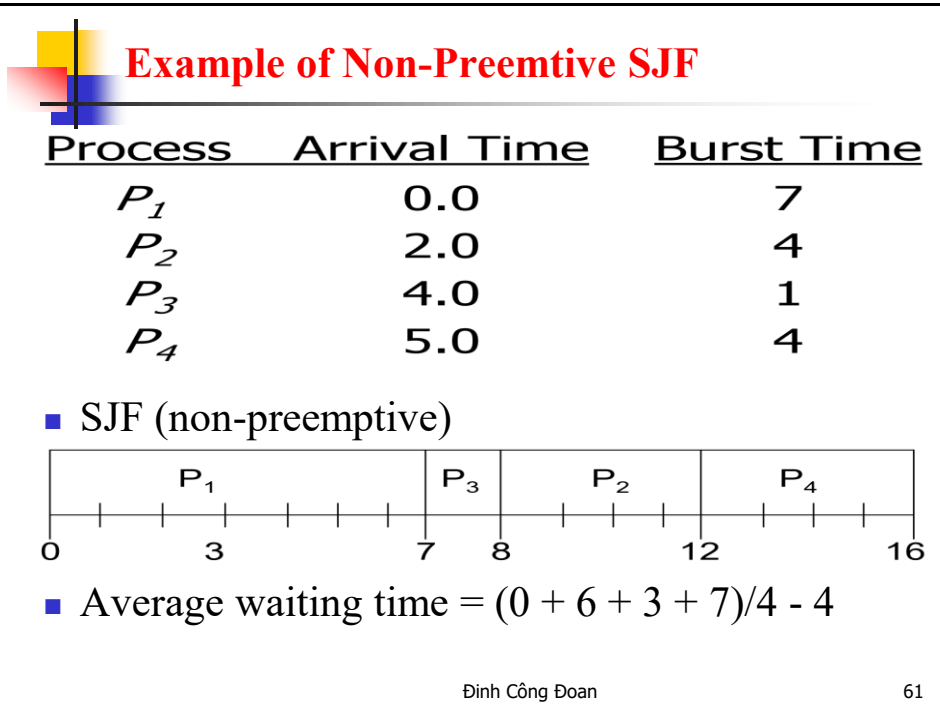




Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest time.
- Two Schemes:
 - ✓ Scheme 1: Non-preemptive
 - Once CPU is given to the process it cannot be preempted until it completes its CPU burst.
 - ✓ Scheme 2: Preemptive
 - If a new CPU process arrives with CPU burst length less than remaining time of current executing process, preempt. Also called Shortest-Remaining-Time-First (SRTF).
 - ✓ SJF is optimal - gives minimum average waiting time for a given set of processes.

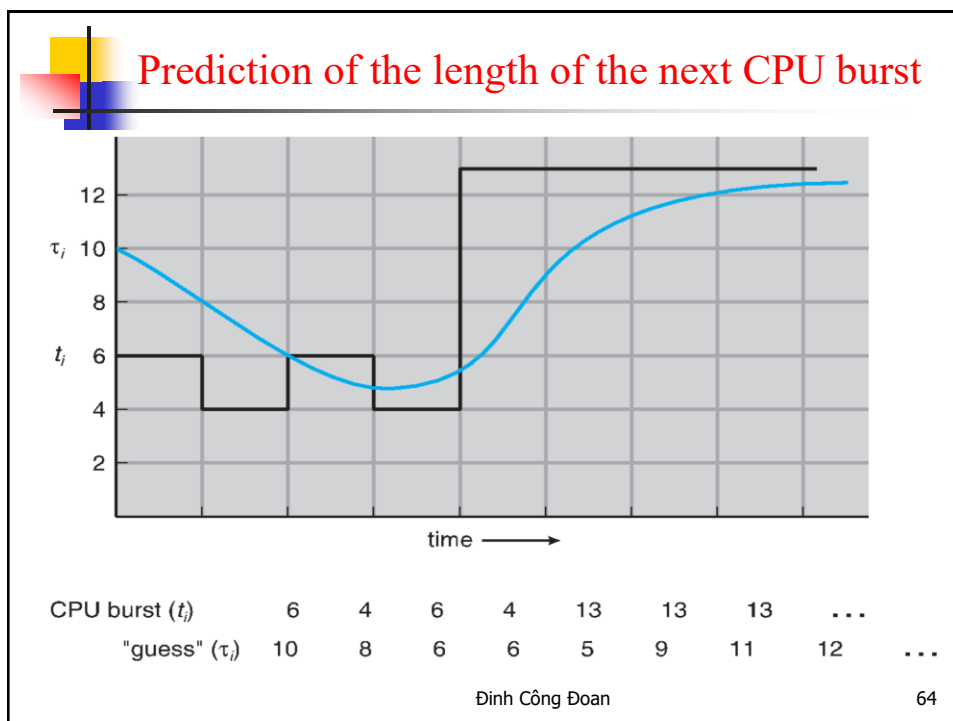
Đinh Công Đoàn 60



Determining length of next CPU Burst

- One can only estimate the length of burst.
 - ✓ Halting problem: cannot (in general) determine if a program will run forever on some input or complete in finite time
- Estimate next CPU burst by an exponentially weighted moving average over previous ones:
 - ✓ t_n = actual length of n^{th} burst
 - ✓ τ_{n+1} = predicted value for the next CPU burst
 - ✓ α = weight, $0 \leq \alpha \leq 1$
 - ✓ Define : $\forall \tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n$

Đinh Công Đoàn 63





Exponential Averaging(cont.)

- $\alpha = 0$
 - ✓ $\tau_{n+1} = \tau_n$; Recent history does not count
- $\alpha = 1$
 - ✓ $\tau_{n+1} = t_n$; Only the actual last CPU burst counts.
- Similarly, expanding the formula:
 - ✓ $\tau_{n+1} = \alpha t_n + (1-\alpha) \alpha t_{n-1} + \dots + (1-\alpha)^j \alpha t_{n-j} + \dots + (1-\alpha)^{(n+1)} \tau_0$
 - ✓ Each successive term has less weight than its predecessor

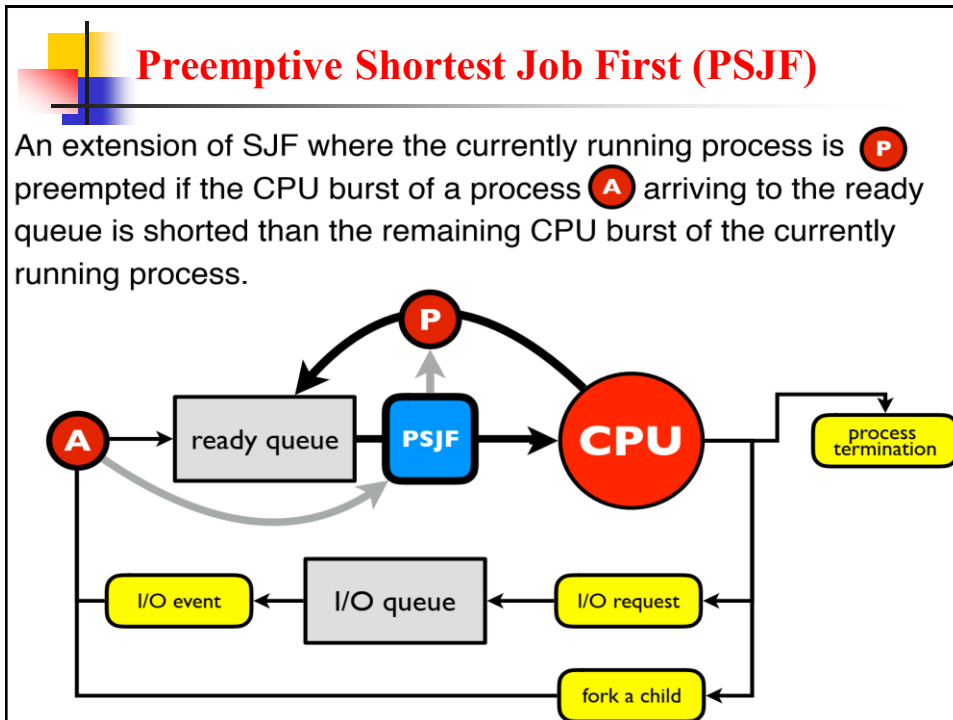
Đinh Công Đoàn 65



Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer $\equiv$ highest priority).
 - ✓ Preemptive
 - ✓ Nonpreemptive
- SJF is a priority scheduling where priority is the predicted next CPU burst time.
- Problem $\equiv$ Starvation – low priority processes may never execute.
- Solution $\equiv$ Aging – as time progresses increase the priority of the process.

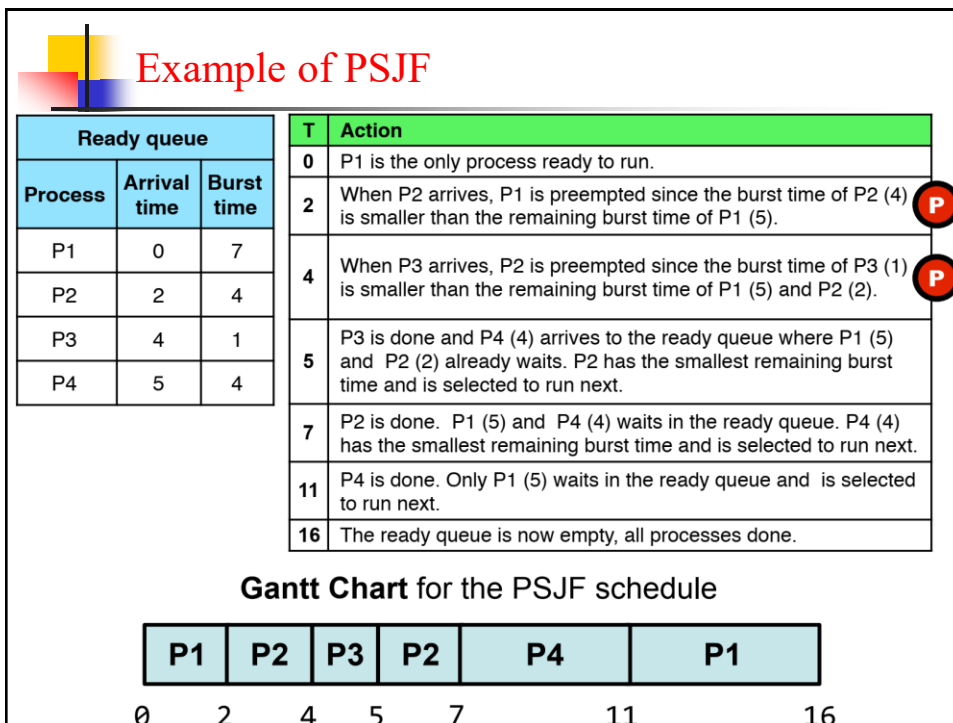
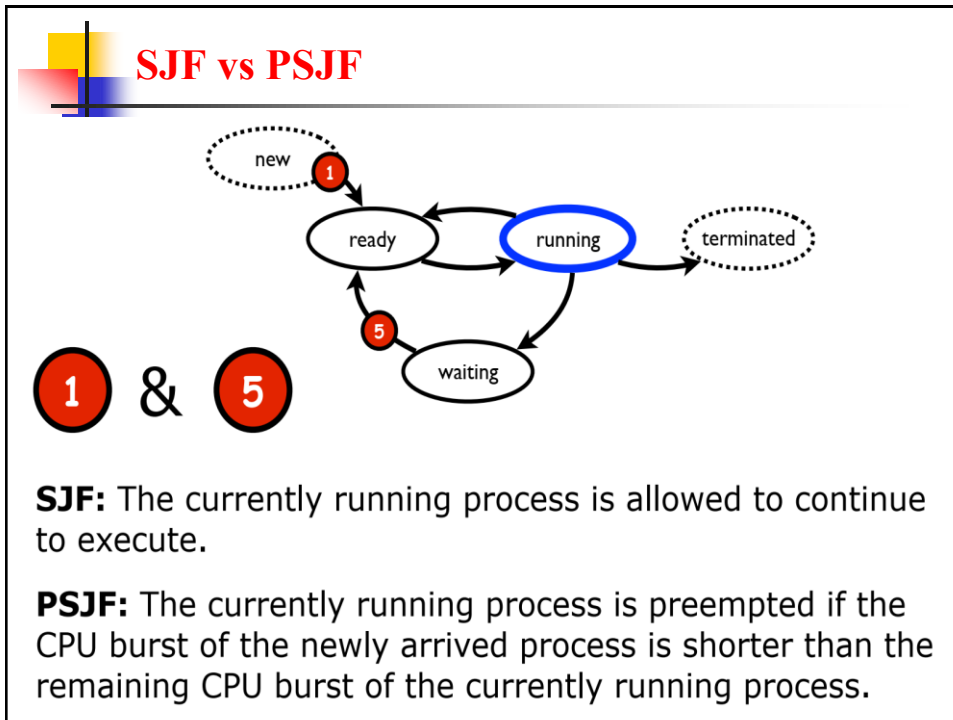
Đinh Công Đoàn 66



Preemptive Shortest Job First (PSJF)

- The currently running process is preempted if the CPU burst of a process arriving to the ready queue is shorter than the remaining CPU burst of the currently running process.
 - ✓ Also known as shortest remaining time first (SRTF).
 - ✓ The currently executing process will always run until completion or until a new process is added to the ready queue that requires a smaller amount of time to complete.
 - ✓ Shortest remaining time is advantageous because **short processes are handled very quickly**.
 - ✓ Requires very **little overhead** since a decision is made only when a process completes or a new process is added, and when a new process is added the algorithm only needs to compare the currently executing process with the new process, ignoring all other processes currently in the ready queue.

Đinh Công Đoàn
68



Average waiting time

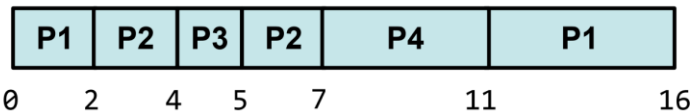
READY QUEUE			PSJF			
Process	Arrival time	Burst time	$T_{dispatch}$	-	$T_{arrival}$	= Waiting time
P1	0	7	0	-	0	= 0
			11	-	2	= 9
P2	2	4	2	-	2	= 0
			5	-	4	= 1
P3	4	1	4	-	4	= 0
P4	5	4	7	-	5	= 2

Processes may have to wait in the ready queue more than once.

Average waiting time

$$(9 + 1 + 0 + 2)/4 = 12/4 = 3$$

Gantt Chart for the PSJF schedule



- **SJF** gives the optimal average waiting time for a given set of processes currently in the ready queue.
- **PSJF** aims at decreasing the average waiting time by allowing a newly arriving process to preempt the currently running process if the CPU burst of the new process is shorter than what remains of the currently running process

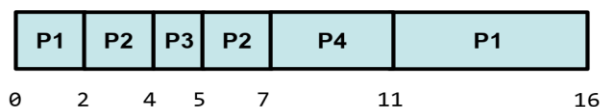
SJF vs PSJF

SJF, average waiting time = 4

READY QUEUE		
Process	Arrival time	Burst Time
P1	0	7
P2	2	4
P3	4	1
P4	5	4



PSJF, average waiting time = 3



Priority scheduling

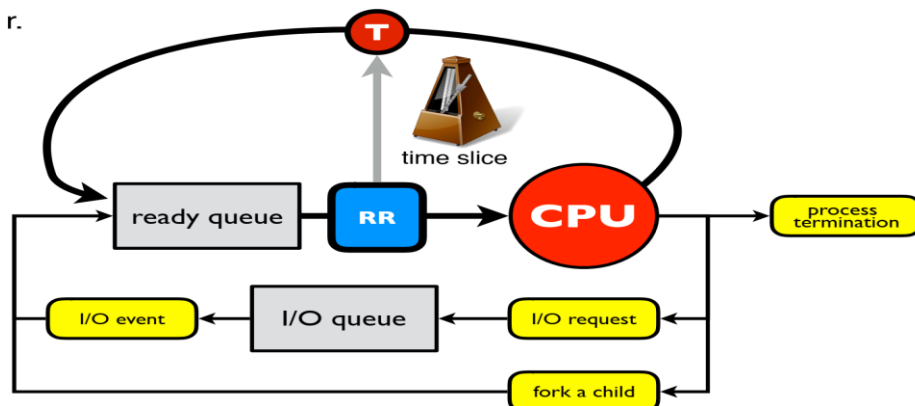
- A priority number (integer) is associated with each process. The CPU is allocated to the process with the highest priority (smallest integer = highest priority).
 - ✓ **Preemption** – should a higher priority process be allowed to preempt a running process with lower priority?
 - ✓ **Starvation** – low priority processes may never execute.
 - ✓ **Ageing** – ensure that jobs with lower priority will eventually complete their execution. Ageing can be implemented by increasing the priority of a process as time progresses.
 - ✓ **SJF** is a priority scheduling algorithm where priority is the predicted next CPU burst time.

Đinh Công Đoàn

73

Round Robin (RR)

- Round Robin (RR) is a scheduling algorithm where time slices are assigned to each process in equal portions and in circular order



Đinh Công Đoàn

74



Round Robin (RR)

- Each process gets a small unit of CPU time (time quantum), usually 10-100 milliseconds. After this time has elapsed, the process is **preempted** and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once

Đinh Công Đoàn

75



Response time and extreme behaviours (RR)

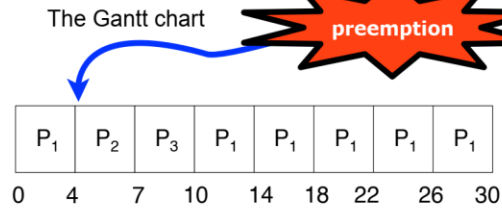
- A nice property of RR is that there is an upper bound the response time.
- **Upper bound for response time:** $(n - 1)q$ time units.
- **Extreme behaviours**
 - ✓ q large $\Rightarrow$ FCFS/FIFO
 - ✓ q small $\Rightarrow$ q must be large with respect to context switch, otherwise the overhead is too high.

Đinh Công Đoàn

76

Example of RR with Time Quantum = 4

READY QUEUE	
Process	Burst Time
P1	24
P2	3
P3	3



Turnaround time

Amount of time to execute a particular process.

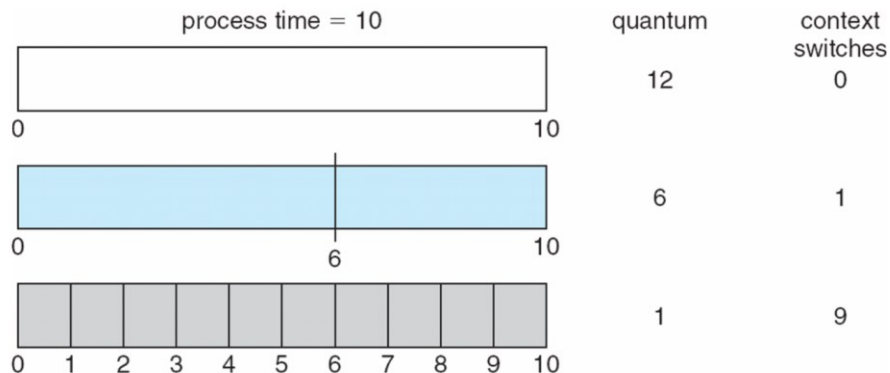
Response time

Amount of time it takes from when a request was submitted until the first response is produced.

Round Robin typically have **higher** average **turnaround** times than **SJF**, but **better response times**.

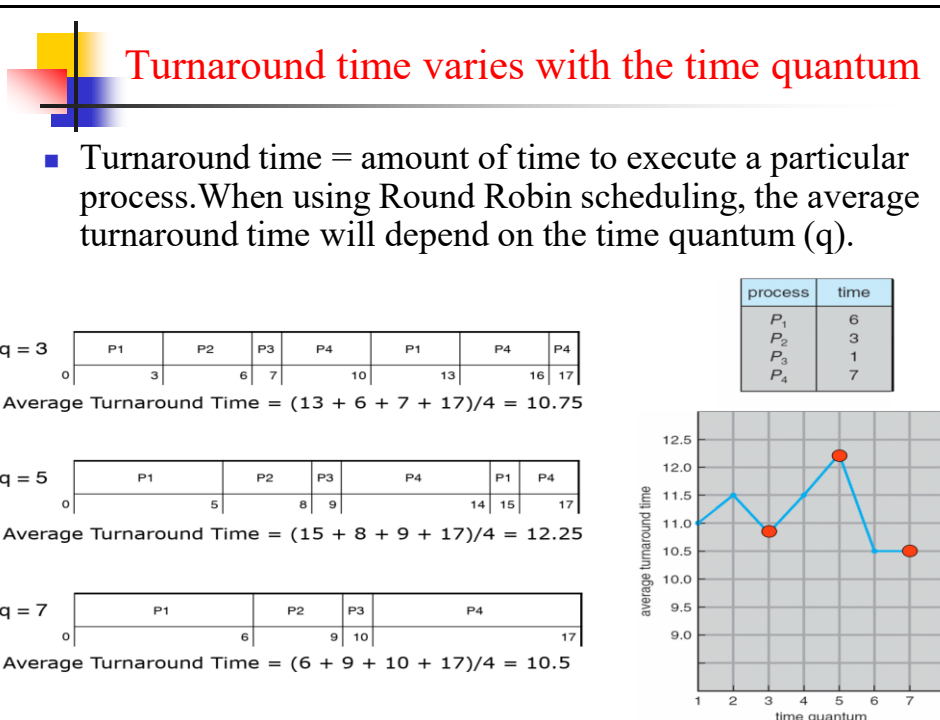
Time Quantum and Context Switch Time

- The RR time quantum (q) affects the number of context switches for a process.



Đinh Công Đoàn

78



Round Robin (RR)

- Each process gets a small unit of CPU time
 - ✓ Time quantum usually 10-100 milliseconds.
 - ✓ After this time has elapsed, the process is preempted, moved to the end of the ready queue, and the process at the head of the ready queue is allocated the CPU.
- n processes, time quantum = q
 - ✓ Each process gets $1/n$ CPU time in chunks of at most q at a time.
 - ✓ No process waits more than $(n-1)q$ time units.
 - ✓ Performance considerations:
 - Time slice q too large – FIFO (FCFS) behavior
 - Time slice q too small - Overhead of context switch is too expensive.
 - Heuristic - 70-80% of jobs block within time slice

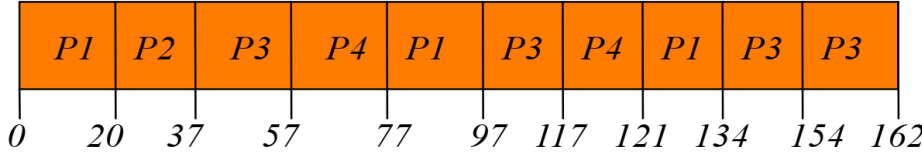
Đinh Công Đoàn
80



Round Robin Example

- **Time Quantum = 20**
- *Gantt Chart for Schedule*

Process	Burst Time
P1	53
P2	17
P3	68
P4	24



- *Typically, higher average turnaround time than SRTF, but better response time*

Đinh Công Đoàn
81



Characteristics of CPU scheduling algorithms

Metric	Description
Performance	A context dependent metric. What do we mean by performance?
Waiting time	Amount of time a process has been waiting in the ready queue.
Turn around time	Amount of time to execute a particular process.
Response time	Amount of time it takes from when a request was submitted until the first response is produced.



Foreground and background processes

- Sometimes process can easily be classified into two groups, one set of processes that interacts with users and one set that doesn't.
- **Background process (batch)**
A process that don't interacts with any user is called a background process or a batch process.
- **Foreground process (interactive)**
A process that interacts with users is called a foreground process or an interactive process.

Đinh Công Đoàn

83



Multilevel queue scheduling

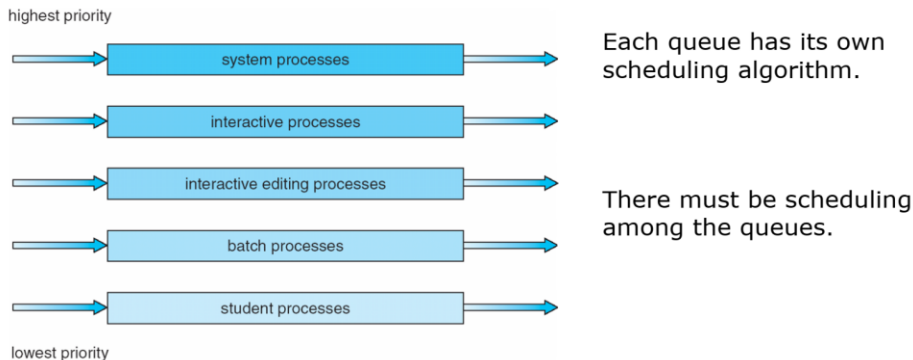
- A multi-level queue scheduling algorithm is used in scenarios where the processes can be classified into groups based on properties like process type, CPU time, IO access, memory size, etc
- General classification of processes:
 - ✓ foreground (interactive)
 - ✓ background (batch)
- In a multi-level queue scheduling algorithm, there will be **n** number of **queues**, where **n** is the number of groups the processes are classified into.
 - ✓ Each queue will be assigned a priority and will have its own scheduling algorithm like Round-robin scheduling or FCFS.

Đinh Công Đoàn

84

Multilevel queue scheduling

- Use several ready queues. A process is permanently assigned to one queue, generally based on some property of the process, such as memory size, process priority, or process type.
- For example, the foreground queue may have absolute priority over the background queue. If an interactive process enters the ready queue while a batch process is running, the batch process will be preempted.



Đinh Công Đoàn

85

Multilevel queue scheduling

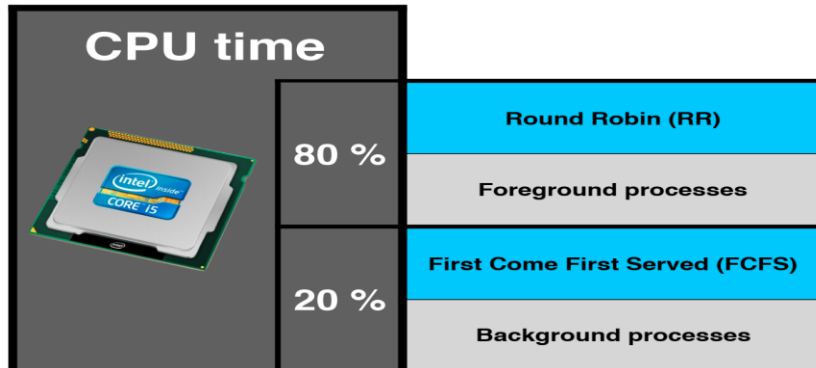
- How to select scheduling algorithms for the various queues?
- Ready queue is partitioned into separate queues:
 - ✓ foreground (interactive)
 - ✓ background (batch)
- Each queue has its own scheduling algorithm
 - ✓ foreground – RR
 - ✓ background – FCFS
- Scheduling must be done between the queues
 - ✓ **Fixed priority scheduling**; (i.e., serve all from foreground then from background). Possibility of starvation.
 - ✓ **Time slice** – each queue gets a certain amount of CPU time which it can schedule amongst its processes.

Đinh Công Đoàn

86

Multilevel queue scheduling

- Example of time slicing among multilevel queues. Foreground processes are given 80 % of the CPU time for RR scheduling and background processes are given 20 % of the CPU time for FCFS scheduling.



Đinh Công Đoàn

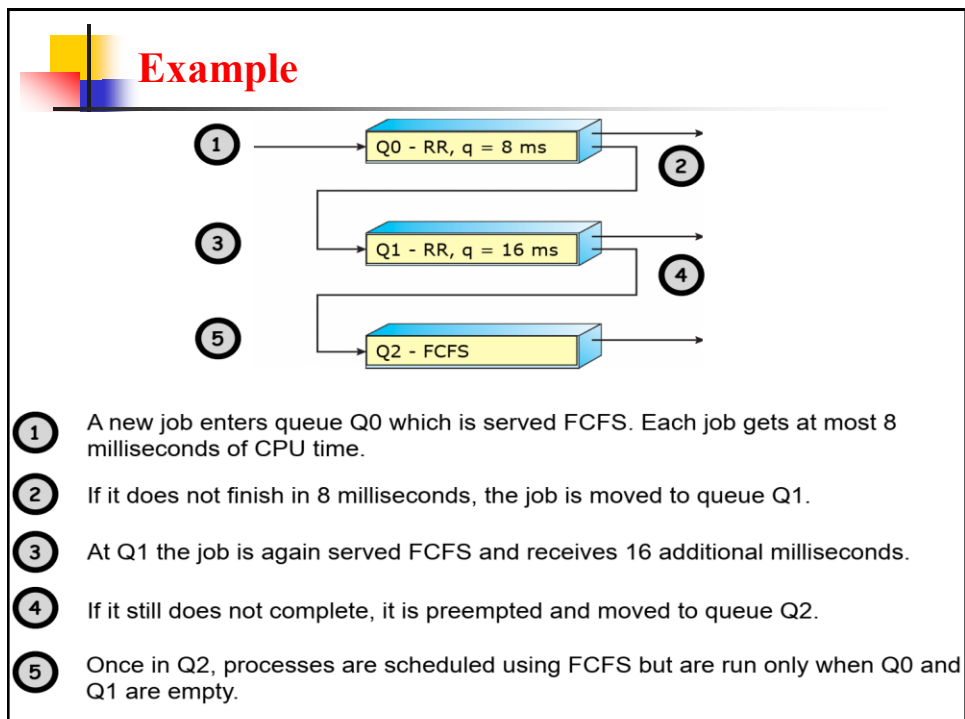
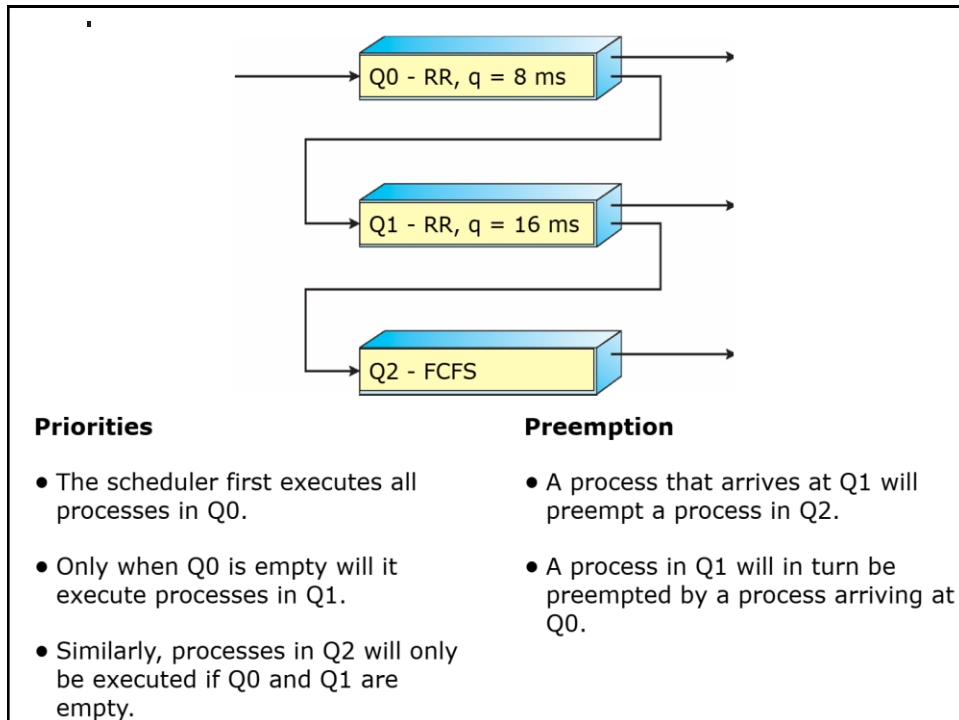
87

Multilevel feedback queue scheduling

- The idea is to separate processes according to the characteristics of their CPU bursts. If a process uses too much CPU time it will be moved to a lower-priority queue.
- Design objectives
 - ✓ Give preference to short CPU bursts.
 - ✓ Give preference to I/O bound processes.
 - ✓ Separate processes into categories based on their need for the CPU

Đinh Công Đoàn

88





Multilevel Queue

- Ready Queue partitioned into separate queues
 - ✓ Example: system processes, foreground (interactive), background (batch), student grades
- Each queue has its own scheduling algorithm
 - ✓ Example: foreground (RR), background (FCFS)
- Processes assigned to one queue permanently.
- Scheduling must be done between the queues
 - ✓ Fixed priority - serve all from foreground, then from background. Possibility of starvation.
 - ✓ Time slice - Each queue gets some CPU time that it schedules -e.g. 80% foreground (RR), 20% background (FCFS)

Đinh Công Đoàn

91



Multilevel Feedback Queue

- Multilevel Queue with priorities
- A process can move between the queues.
 - ✓ Aging can be implemented this way.
- Parameters for a multilevel feedback queue scheduler:
 - ✓ number of queues.
 - ✓ scheduling algorithm for each queue.
 - ✓ method used to determine when to upgrade a process.
 - ✓ method used to determine when to demote a process.
 - ✓ method used to determine which queue a process will enter when that process needs service

Đinh Công Đoàn

92

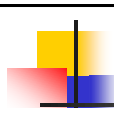


Multilevel Feedback Queues

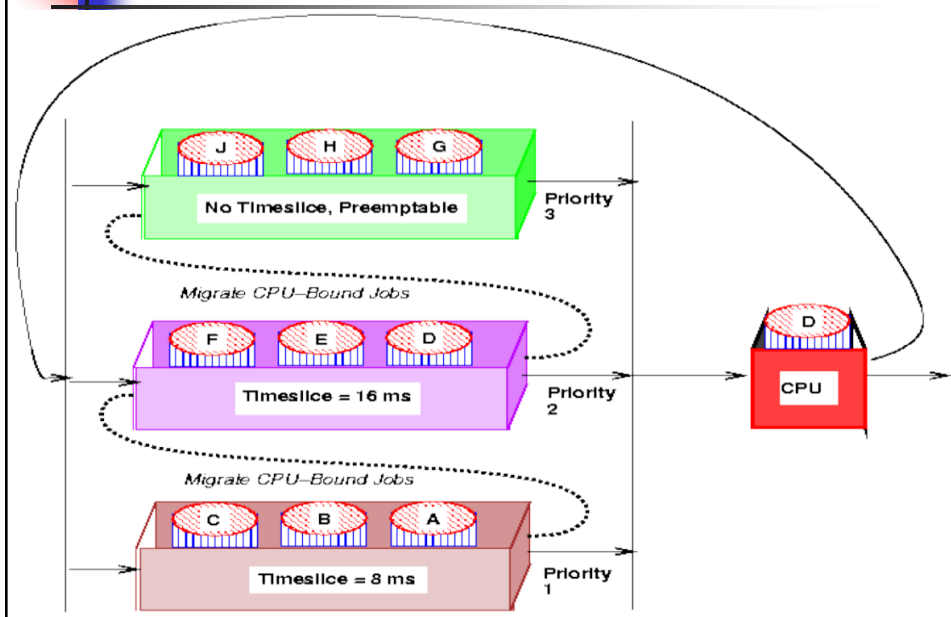
- Example: Three Queues –
 - ✓ Q0 - time quantum 8 milliseconds (FCFS)
 - ✓ Q1 - time quantum 16 milliseconds (FCFS)
 - ✓ Q2 – FCFS
- Scheduling
 - ✓ New job enters Q0 - When it gains CPU, it receives 8 milliseconds. If job does not finish, move it to Q1.
 - ✓ At Q1, when job gains CPU, it receives 16 more milliseconds. If job does not complete, it is preempted and moved to Q2

Đinh Công Đoàn

93



Multilevel Feedback Queues





Multiple-Processor Scheduling

- CPU scheduling becomes more complex when multiple CPUs are available.
 - ✓ Have one ready queue accessed by each CPU.
 - Self scheduled - each CPU dispatches a job from ready Q
 - Manager-worker - one CPU schedules the other CPUs
- Homogeneous processors within multiprocessor.
 - ✓ Permits Load Sharing
- Asymmetric multiprocessing
 - ✓ only 1 CPU runs kernel, others run user programs

Đinh Công Đoàn 95



Real-Time Scheduling

- Hard Real-time Computing –
 - ✓ required to complete a critical task within a guaranteed amount of time.
- Soft Real-time Computing –
 - ✓ requires that critical processes receive priority over less important ones.
- Types of real-time Schedulers
 - ✓ Periodic Schedulers - Fixed Arrival Rate
 - ✓ Demand-Driven Schedulers - Variable Arrival Rate
 - ✓ Deadline Schedulers - Priority determined by deadline

Đinh Công Đoàn 96



Issues in Real-time Scheduling

- Dispatch Latency
 - ✓ Problem - Need to keep dispatch latency small, OS may enforce process to wait for system call or I/O to complete.
 - ✓ Solution - Make system calls preemptible, determine safe criteria such that kernel can be interrupted.
- Priority Inversion and Inheritance
 - ✓ Problem: Priority Inversion
 - Higher Priority Process P0 needs kernel resource currently being used by another lower priority process P1.
 - P0 must wait for P1, but P1 isn't getting scheduled in CPU!
 - ✓ Solution: Priority Inheritance
 - Low priority process now inherits high priority until it has completed use of the resource in question.

Đinh Công Đoàn 97



Scheduling Algorithm Evaluation

- Deterministic Modeling
 - ✓ Takes a particular predetermined workload and defines the performance of each algorithm for that workload.
 - ✓ Too specific, requires exact knowledge to be useful.
- Queuing Models and Queuing Theory
 - ✓ Use distributions of CPU and I/O bursts. Knowing arrival and service rates - can compute utilization, average queue length, average wait time, etc...
 - ✓ Little's formula: $n = \lambda \times W$
 - n is the average queue length, λ is the avg. arrival rate and W is the avg. waiting time in queue.
- Other techniques: Simulations, Implementation

Đinh Công Đoàn 98